EXPERIMENT 1

Breadth-First Search for Hospital Route Planning

AIM

To find the shortest route between two locations in a hospital layout using Breadth-First Search (BFS).

QUESTION

Develop a Python-based navigation system for an emergency response team operating inside a hospital represented as an unweighted graph. Rooms and junctions are vertices, while accessible corridors are edges. Implement Breadth-First Search (BFS) to identify the route from the emergency entrance to a designated critical-care unit.

The program must:

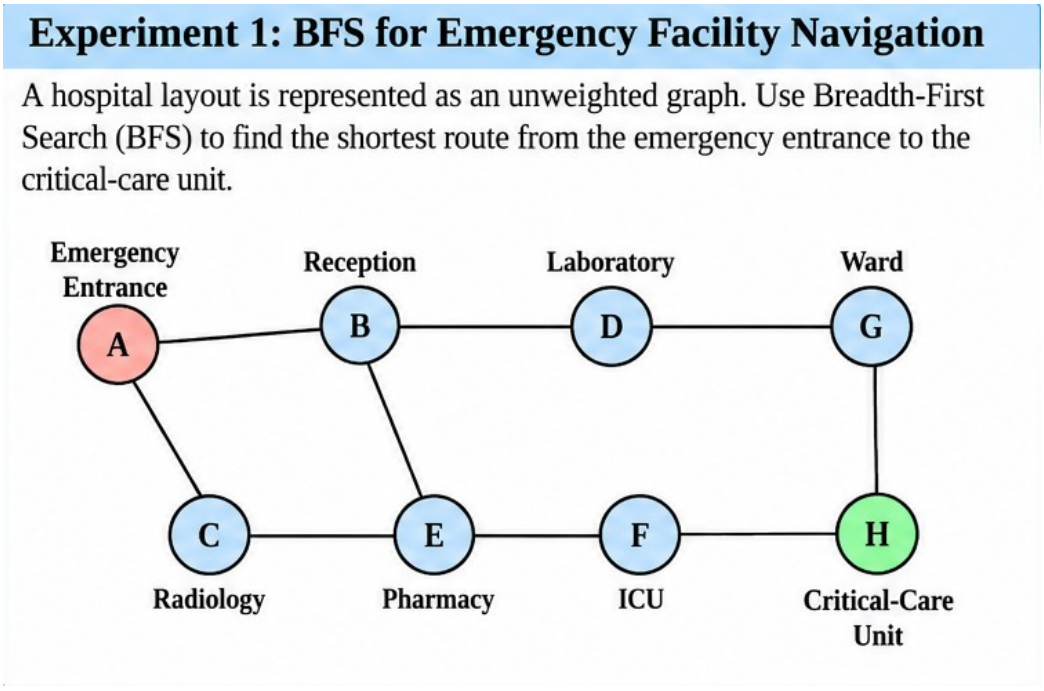
1. Represent the hospital layout as an adjacency list.

2. Accept the source and destination nodes.

3. Find and display the shortest route in terms of the number of corridors traversed.

4. Report the number of nodes explored and explain how BFS guarantees a shortest path in an unweighted graph.

DIAGRAM – EXPERIMENT 1



PROGRAM

```
from collections import deque
graph = {
    'A': ['B', 'C'],
```

```
    'B': ['A', 'D', 'E'],
    'C': ['A', 'E'],
    'D': ['B', 'G'],
    'E': ['B', 'C', 'F'],
    'F': ['E', 'H'],
    'G': ['D', 'H'],
    'H': ['F', 'G']
}
def bfs(graph, start, goal):
    queue = deque([start])
    visited = {start}
    parent = {start: None}
    distance = {start: 0}
    order = []
    print("BFS: Hospital Navigation")
    print(f"Start: {start} | Goal: {goal}\n")
    while queue:
        node = queue.popleft()
        order.append(node)
        print(f"Step {len(order)}: Node = {node}")
        print(f"  Queue: {list(queue)}")
        if node == goal:
            print("  Goal reached!")
            break
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = node
                distance[neighbor] = distance[node] + 1
                queue.append(neighbor)
                print(f"  {node} -> {neighbor}: Added to queue")
            else:
                print(f"  {node} -> {neighbor}: Already visited")
        print(f"  Visited: {sorted(visited)}")
    if goal not in parent:
        print("\nNo route found.")
        return
    path = []
    current = goal
    while current is not None:
        path.append(current)
        current = parent[current]
    path.reverse()
    print("\n--- Final Result ---")
    print("Traversal order:", " -> ".join(order))
    print("Shortest path  :", " -> ".join(path))
    print("Path cost      :", distance[goal], "corridors")
    print("Nodes explored :", len(order))
bfs(graph, 'A', 'H')
```

OUTPUT

```
BFS: Hospital Navigation
Start: A | Goal: H
Step 1: Node = A
  Queue: []
  A -> B: Added to queue
  A -> C: Added to queue
  Visited: ['A', 'B', 'C']
Step 2: Node = B
  Queue: ['C']
  B -> A: Already visited
  B -> D: Added to queue
  B -> E: Added to queue
  Visited: ['A', 'B', 'C', 'D', 'E']
Step 3: Node = C
  Queue: ['D', 'E']
  C -> A: Already visited
  C -> E: Already visited
  Visited: ['A', 'B', 'C', 'D', 'E']
Step 4: Node = D
  Queue: ['E']
  D -> B: Already visited
  D -> G: Added to queue
  Visited: ['A', 'B', 'C', 'D', 'E', 'G']
Step 5: Node = E
  Queue: ['G']
  E -> B: Already visited
  E -> C: Already visited
  E -> F: Added to queue
  Visited: ['A', 'B', 'C', 'D', 'E', 'F', 'G']
Step 6: Node = G
  Queue: ['F']
  G -> D: Already visited
  G -> H: Added to queue
  Visited: ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']
Step 7: Node = F
  Queue: ['H']
  F -> E: Already visited
  F -> H: Already visited
  Visited: ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']
Step 8: Node = H
  Queue: []
  Goal reached!
--- Final Result ---
Traversal order: A -> B -> C -> D -> E -> G -> F -> H
Shortest path  : A -> B -> D -> G -> H
Path cost      : 4 corridors
Nodes explored : 8
```

RESULT

The Breadth-First Search (BFS) program found the route A → B → D → G → H from the emergency entrance to the destination. The route traverses 4 corridors, and 8 nodes were explored before the destination was reached.

EXPERIMENT 2

Depth-First Search for Dependency and Reachability Analysis

AIM

To analyze task dependencies and determine whether a target task is reachable from a starting task using Depth-First Search (DFS).

QUESTION

An AI deployment pipeline consists of interconnected processing tasks, where each task depends on one or more downstream tasks. Model the pipeline as a directed graph and implement Depth-First Search (DFS) in Python to determine whether a specified target task is reachable from a given starting task.

The program must:

1. Represent the pipeline using a directed adjacency list.

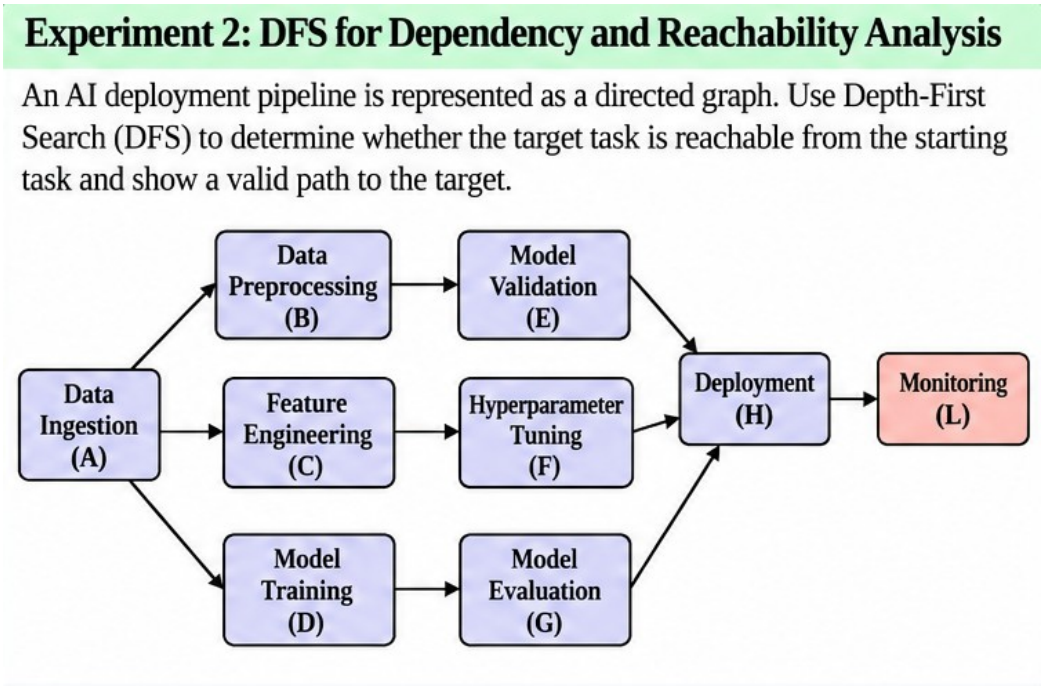
2. Accept a starting task and a target task.

3. Display the DFS traversal order and determine whether the target is reachable.

4. Reconstruct a valid path to the target, if one exists.

5. Explain how visited-node tracking prevents repeated exploration and how DFS behaves when the graph contains cycles.

DIAGRAM – EXPERIMENT 2



PROGRAM

```
# Experiment 2: DFS for Dependency and Reachability Analysis

# Directed adjacency list representing the AI pipeline
graph = {
    'A': ['B', 'C', 'D'],
    'B': ['E'],
    'C': ['F'],
    'D': ['G'],
    'E': ['H'],
    'F': ['H'],
    'G': ['H'],
    'H': ['L'],
    'L': []
}

def dfs(graph, start, target):
    visited = set()
    traversal = []
    parent = {start: None}

    def explore(node):
        visited.add(node)
        traversal.append(node)

        if node == target:
            return True

        for neighbour in graph.get(node, []):
            if neighbour not in visited:
                parent[neighbour] = node

                if explore(neighbour):
                    return True

        return False

    found = explore(start)

    print("\nDFS Traversal Order:", " -> ".join(traversal))

    if found:
        path = []
        node = target

        while node is not None:
            path.append(node)
            node = parent[node]
```

```
        path.reverse()

        print("Target Reachable: Yes")
        print("Valid Path:", " -> ".join(path))
    else:
        print("Target Reachable: No")
        print("No valid path exists.")

# Accept input from the user
start = input("Enter starting task (A, B, C, D, E, F, G, H, L): ").upper()
target = input("Enter target task (A, B, C, D, E, F, G, H, L): ").upper()

if start not in graph or target not in graph:
    print("Invalid task! Please enter a valid task label.")
else:
    dfs(graph, start, target)
```

OUTPUT

```
Enter starting task (A, B, C, D, E, F, G, H, L): A
Enter target task (A, B, C, D, E, F, G, H, L): L

DFS Traversal Order: A -> B -> E -> H -> L
Target Reachable: Yes
Valid Path: A -> B -> E -> H -> L
```

RESULT

The Depth-First Search (DFS) program successfully reached task L from task A. The traversal order and valid path were A → B → E → H → L, confirming that the target task is reachable.

EXPERIMENT 3

Water Jug Problem Using Breadth-First Search

AIM

To solve the water jug problem by finding a sequence of valid operations that measures the required quantity of water.

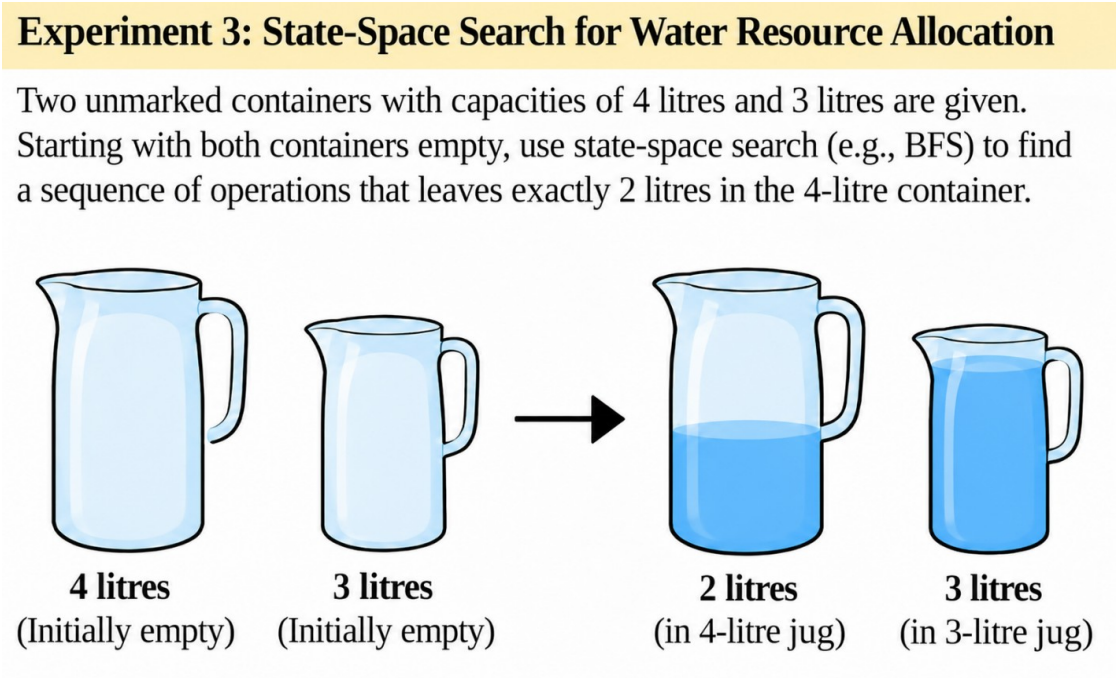
QUESTION

Design a Python-based state-space search solution for a water allocation problem involving two unmarked containers with capacities of 4 litres and 3 litres. Starting with both containers empty, determine a sequence of valid operations that leaves exactly 2 litres in the 4-litre container.

The program must:

1. Represent each state as the quantities of water in both containers.
2. Define the legal actions: fill, empty and transfer water between containers.
3. Search the state space using BFS and identify a valid solution.
4. Display the sequence of states and actions from the initial state to the goal.
5. Explain how state representation, goal testing and repeated-state detection affect the search.

DIAGRAM – EXPERIMENT 3



PROGRAM

```
# Experiment-3:
from collections import deque

# Capacities of the two containers
```

```
CAP_A = 4
CAP_B = 3

# State: (water in 4-litre container, water in 3-litre container)
INITIAL_STATE = (0, 0)
GOAL_AMOUNT = 2


def get_next_states(state):
    a, b = state
    next_states = []

    def add(action, new_a, new_b):
        new_state = (new_a, new_b)
        if new_state != state:
            next_states.append((action, new_state))

    # Fill either container
    add("Fill 4-litre container", CAP_A, b)
    add("Fill 3-litre container", a, CAP_B)

    # Empty either container
    add("Empty 4-litre container", 0, b)
    add("Empty 3-litre container", a, 0)

    # Pour from 4-litre container into 3-litre container
    transfer = min(a, CAP_B - b)
    add("Pour 4L into 3L", a - transfer, b + transfer)

    # Pour from 3-litre container into 4-litre container
    transfer = min(b, CAP_A - a)
    add("Pour 3L into 4L", a + transfer, b - transfer)

    return next_states


def bfs():
    queue = deque([INITIAL_STATE])
    visited = {INITIAL_STATE}
    parent = {INITIAL_STATE: (None, None)}

    while queue:
        state = queue.popleft()
        a, b = state

        # Goal test
        if a == GOAL_AMOUNT:
            solution = []
            current = state
```

```
            while parent[current][0] is not None:
                previous, action = parent[current]
                solution.append((action, current))
                current = previous

            solution.reverse()
            return solution

        for action, next_state in get_next_states(state):
            if next_state not in visited:
                visited.add(next_state)
                parent[next_state] = (state, action)
                queue.append(next_state)

    return None


# Execute BFS
solution = bfs()

if solution is None:
    print("No solution exists.")
else:
    print("Initial State: (0, 0)")
    print("State format: (4-litre container, 3-litre container)")
    print()

    current_state = INITIAL_STATE

    for step, (action, next_state) in enumerate(solution, start=1):
        print(f"Step {step}: {action}")
        print(f"        {current_state} -> {next_state}")
        current_state = next_state

    print("\nGoal reached!")
    print(f"4-litre container contains {current_state[0]} litres.")
    print(f"Total operations: {len(solution)}")
```

OUTPUT

```
Initial State: (0, 0)
State format: (4-litre container, 3-litre container)

Step 1: Fill 4-litre container
        (0, 0) -> (4, 0)
Step 2: Pour 4L into 3L
        (4, 0) -> (1, 3)
Step 3: Empty 3-litre container
        (1, 3) -> (1, 0)
```

Step 4: Pour 4L into 3L
(1, 0) -> (0, 1)
Step 5: Fill 4-litre container
(0, 1) -> (4, 1)
Step 6: Pour 4L into 3L
(4, 1) -> (2, 3)

Goal reached!
4-litre container contains 2 litres.
Total operations: 6

RESULT

The BFS state-space search successfully measured 2 litres in the 4-litre container. It reached the goal state (2, 3) from the initial state (0, 0) in 6 operations.

EXPERIMENT 4

A* Search for Intelligent Route Planning

AIM

To find a minimum-cost route between two locations in a weighted delivery network using the A* search algorithm.

QUESTION

Develop an intelligent route-planning system for an autonomous delivery robot navigating a network of locations connected by paths with different travel costs. Implement the A* search algorithm in Python to find a minimum-cost route from a specified starting location to a delivery destination.

The program must:

1. Represent the environment as a weighted graph.

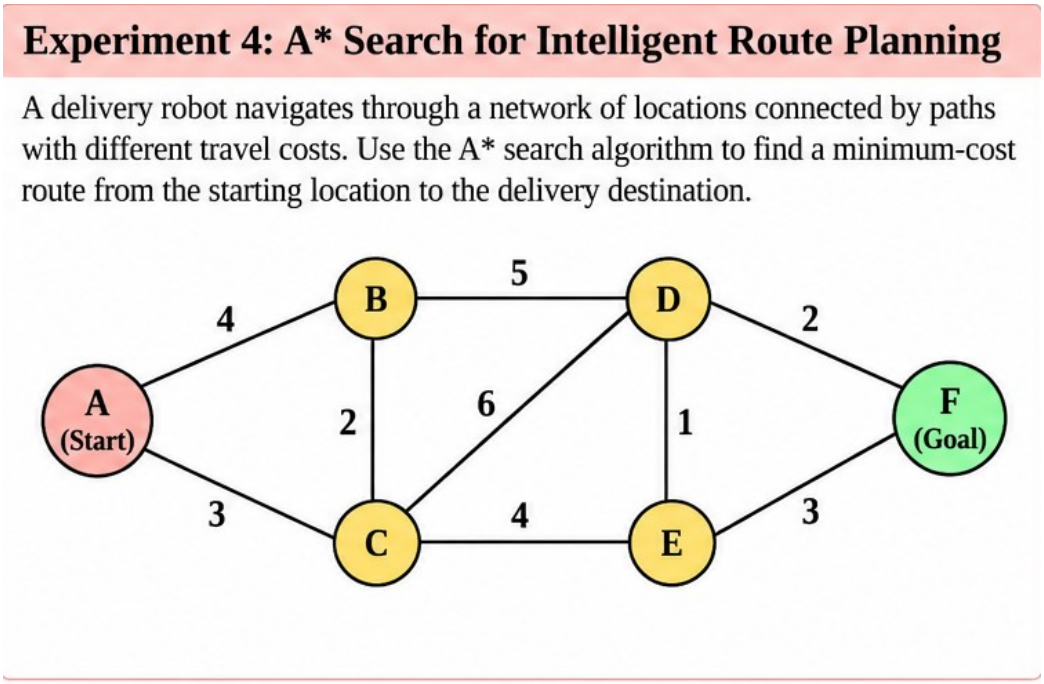
2. Assign a heuristic estimate h(n) to each node and calculate the accumulated path cost g(n).

3. Use f(n) = g(n) + h(n) to determine which node to explore next.

4. Display the selected route, total path cost and node expansion order.

5. Explain how heuristic quality affects search efficiency and why an admissible heuristic matters for optimality.

DIAGRAM – EXPERIMENT 4



PROGRAM

```
# Experiment-4:
import heapq

# Weighted graph representing the delivery network
graph = {
    'A': {'B': 4, 'C': 3},
    'B': {'A': 4, 'C': 2, 'D': 5},
    'C': {'A': 3, 'B': 2, 'D': 6, 'E': 4},
    'D': {'B': 5, 'C': 6, 'E': 1, 'F': 2},
    'E': {'C': 4, 'D': 1, 'F': 3},
    'F': {'D': 2, 'E': 3}
}

# Heuristic estimates for reaching the goal
heuristic = {
    'A': 6,
    'B': 4,
    'C': 5,
    'D': 2,
    'E': 3,
    'F': 0
}


def a_star(start, goal):
    # Priority queue stores (f, g, node)
    open_list = [(heuristic[start], 0, start)]

    # Lowest known cost from start to each node
    g_cost = {start: 0}

    # Parent pointers for reconstructing the route
    parent = {start: None}

    # Nodes whose expansion is reported
    expanded = []

    while open_list:
        f, g, current = heapq.heappop(open_list)

        # Ignore outdated queue entries
        if g != g_cost.get(current):
            continue

        expanded.append(current)
        print(
            f"Expanding {current}: "
            f"g(n)={g}, h(n)={heuristic[current]}, f(n)={f}"
        )
```

```
        # Goal test
        if current == goal:
            path = []
            node = goal

            while node is not None:
                path.append(node)
                node = parent[node]

            path.reverse()

            print("\nOptimal route:", " -> ".join(path))
            print("Total path cost:", g_cost[goal])
            print("Expansion order:", " -> ".join(expanded))
            return path, g_cost[goal], expanded

        # Explore neighbouring locations
        for neighbour, cost in graph[current].items():
            new_g = g_cost[current] + cost

            if new_g < g_cost.get(neighbour, float('inf')):
                g_cost[neighbour] = new_g
                parent[neighbour] = current
                new_f = new_g + heuristic[neighbour]

                heapq.heappush(
                    open_list, (new_f, new_g, neighbour)
                )

    print("No route exists.")
    return None, float('inf'), expanded


# Get start and destination from the user
start = input("Enter starting location (A-F): ").upper()
goal = input("Enter destination location (A-F): ").upper()

if start not in graph or goal not in graph:
    print("Invalid location. Enter a node from A to F.")
else:
    a_star(start, goal)
```

OUTPUT

```
Enter starting location (A-F): A
Enter destination location (A-F): F
Expanding A: g(n)=0, h(n)=6, f(n)=6
Expanding C: g(n)=3, h(n)=5, f(n)=8
Expanding B: g(n)=4, h(n)=4, f(n)=8
```

Expanding E: g(n)=7, h(n)=3, f(n)=10
Expanding D: g(n)=8, h(n)=2, f(n)=10
Expanding F: g(n)=10, h(n)=0, f(n)=10

Optimal route: A -> C -> E -> F
Total path cost: 10
Expansion order: A -> C -> B -> E -> D -> F

RESULT

The A* search program identified A → C → E → F as the minimum-cost route from A to F, with a total path cost of 10. The reported node expansion order was A → C → B → E → D → F.

EXPERIMENT 5

8-Queens Problem Using Backtracking

AIM

To place eight queens on an 8 × 8 chessboard so that no two queens attack each other, using backtracking.

QUESTION

An automated placement system must position eight components on an 8 × 8 grid such that no two components interfere with one another. Model this as the 8-Queens problem, where each queen represents a component and can attack another queen along the same row, column, or diagonal.

Develop a Python program using the backtracking algorithm to find a valid arrangement of eight queens on the chessboard.

The program must:

1. Represent the chessboard as an 8 × 8 grid and track the positions of the queens.

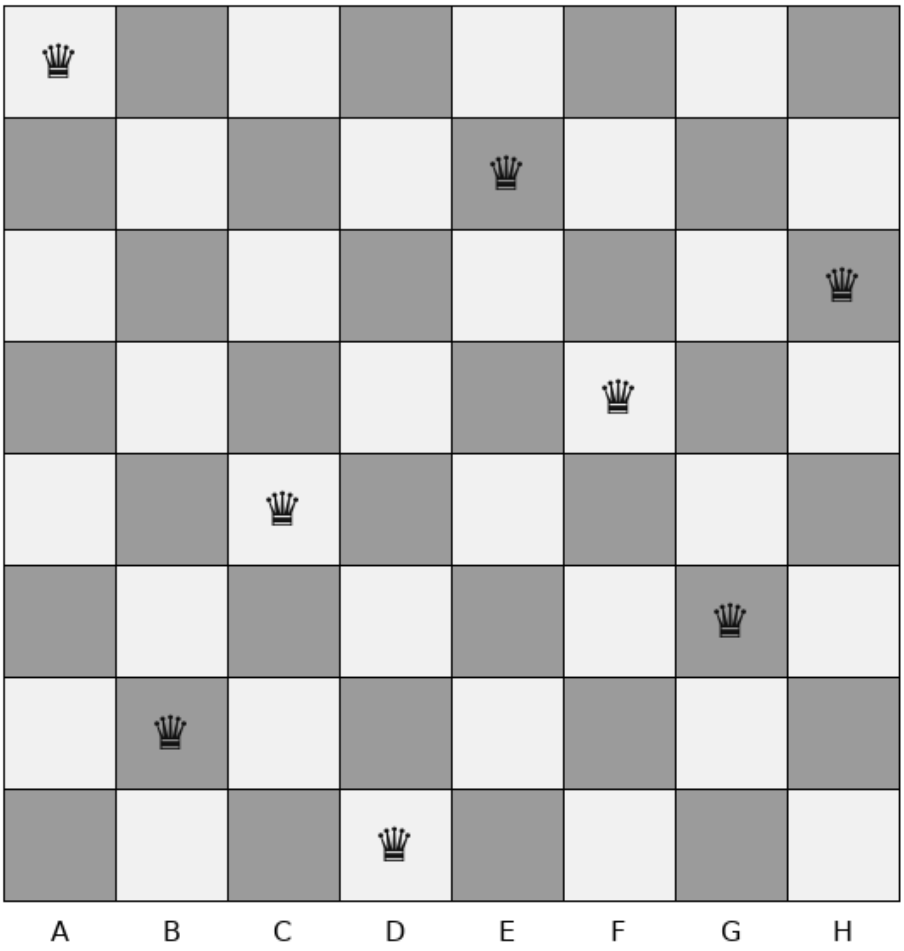
2. Place one queen at a time while ensuring that no two queens share the same row, column, or diagonal.

3. Use recursive backtracking to undo an invalid placement and explore alternative positions.

4. Display the final chessboard arrangement and the positions of all eight queens.

5. Report the number of placement attempts and explain how constraint checking and backtracking help find a valid solution.

DIAGRAM – EXPERIMENT 5



PROGRAM

```
# Experiment 5: 8-Queens Problem Using Backtracking

N = 8
board = [-1] * N
placement_attempts = 0
backtracks = 0


# Check whether a queen can be placed safely
def is_safe(row, col):
    for prev_row in range(row):
        prev_col = board[prev_row]

        # Check same column and diagonal conflicts
        if prev_col == col:
```

```
            return False

        if abs(prev_col - col) == abs(prev_row - row):
            return False

    return True


# Solve using recursive backtracking
def solve(row):
    global placement_attempts, backtracks

    # All eight queens have been placed
    if row == N:
        return True

    # Try every column in the current row
    for col in range(N):
        placement_attempts += 1

        if is_safe(row, col):
            board[row] = col

            if solve(row + 1):
                return True

            # Undo the placement and try another column
            board[row] = -1
            backtracks += 1

    return False


# Display the chessboard
def display_board():
    print("\nFinal Chessboard:")
    print("  " + " ".join(str(c + 1) for c in range(N)))

    for row in range(N):
        print(row + 1, end=" ")
        for col in range(N):
            if board[row] == col:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()


# Execute the program
```

```
if solve(0):
    display_board()

    print("\nQueen Positions (Row, Column):")
    for row in range(N):
        print(f"Queen {row + 1}: ({row + 1}, {board[row] + 1})")

    print("\nPlacement attempts:", placement_attempts)
    print("Backtracks:", backtracks)
    print("Result: Valid arrangement found.")
else:
    print("No solution exists.")
```

OUTPUT

```
Final Chessboard:
  1 2 3 4 5 6 7 8
1 Q . . . . . . .
2 . . . . Q . . .
3 . . . . . . . Q
4 . . . . . Q . .
5 . . Q . . . . .
6 . . . . . . Q .
7 . Q . . . . . .
8 . . . Q . . . .

Queen Positions (Row, Column):
Queen 1: (1, 1)
Queen 2: (2, 5)
Queen 3: (3, 8)
Queen 4: (4, 6)
Queen 5: (5, 3)
Queen 6: (6, 7)
Queen 7: (7, 2)
Queen 8: (8, 4)

Placement attempts: 876
Backtracks: 105
Result: Valid arrangement found.
```

RESULT

The backtracking program found a valid arrangement of all eight queens, with no two queens sharing a column or diagonal. It reported 876 placement attempts and 105 backtracks.

EXPERIMENT 6

Map Colouring Problem Using CSP and Backtracking

AIM

To assign colours to adjacent regions so that no neighbouring regions share the same colour, using CSP and backtracking.

QUESTION

A telecommunications network is divided into interconnected regions. To reduce interference, neighbouring regions must be assigned different frequency bands. Model the network as a Constraint Satisfaction Problem (CSP), where each region is a variable, the available frequency bands form its domain, and neighbouring regions must receive different assignments.

Develop a Python program to solve the map-colouring problem using CSP techniques.

The program must:

1. Represent each region as a variable and define its available colours as a domain.

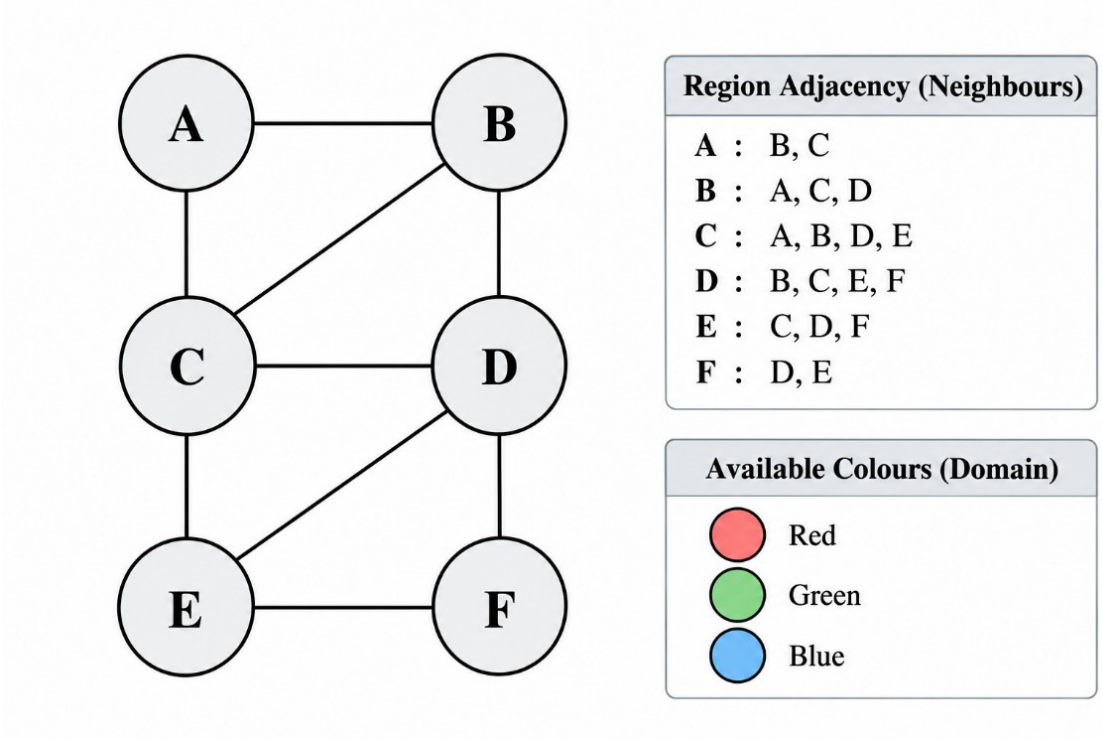
2. Specify adjacency constraints requiring neighbouring regions to have different colours.

3. Use backtracking search to find a valid colour assignment.

4. Display the final map-colouring assignment and verify that all adjacency constraints are satisfied.

5. Report the number of assignments attempted and explain how CSP variables, domains, and constraints guide the search.

DIAGRAM – EXPERIMENT 6



PROGRAM

```
# Experiment-6:
# Map Colouring Problem using CSP and Backtracking

regions = ['A', 'B', 'C', 'D', 'E', 'F']

colours = ['Red', 'Green', 'Blue']

neighbours = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D', 'E'],
    'D': ['B', 'C', 'E', 'F'],
    'E': ['C', 'D', 'F'],
    'F': ['D', 'E']
}

assignment = {}
attempts = 0

def is_valid(region, colour):
    for neighbour in neighbours[region]:
        if neighbour in assignment and assignment[neighbour] == colour:
            return False
    return True

def backtracking():
```

```
    global attempts

    if len(assignment) == len(regions):
        return True

    for region in regions:
        if region not in assignment:
            break

    for colour in colours:
        attempts += 1

        if is_valid(region, colour):
            assignment[region] = colour

            if backtracking():
                return True

            del assignment[region]

    return False

if backtracking():
    print("Valid Colour Assignment:")
    for region in regions:
        print(region, ":", assignment[region])

    print("All adjacency constraints are satisfied.")
else:
    print("No valid colour assignment exists.")

print("Number of assignments attempted:", attempts)
```

OUTPUT

```
Valid Colour Assignment:
A : Red
B : Green
C : Blue
D : Red
E : Green
F : Blue
All adjacency constraints are satisfied.
Number of assignments attempted: 12
```

RESULT

The CSP backtracking program produced a valid colouring using Red, Green, and Blue: A–Red, B–Green, C–Blue, D–Red, E–Green, and F–Blue. All adjacency constraints were satisfied after 12 assignment attempts.

EXPERIMENT 7

Minimax Algorithm for Game-Tree Decision Making

AIM

To determine the best move in a binary game tree using the Minimax algorithm.

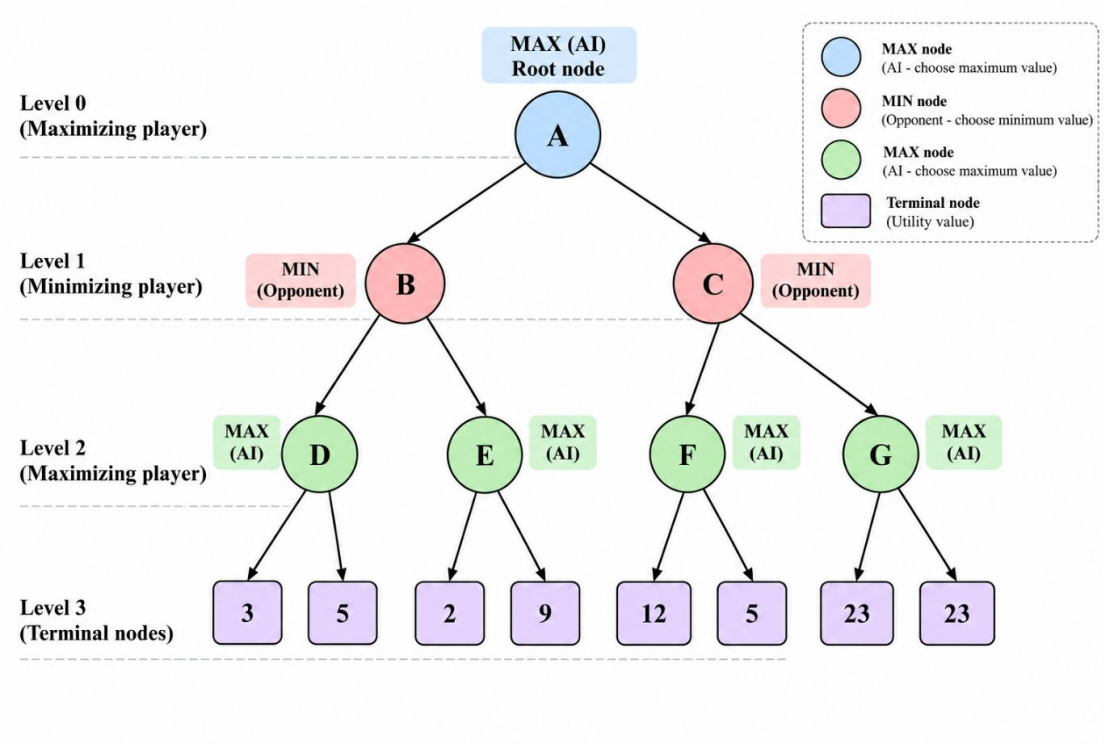
QUESTION

A game-playing AI must select its best move in a two-player game where one player tries to maximize the score while the opponent tries to minimize it.

Develop a Python program to implement the Minimax algorithm for a game represented as a binary game tree. The program must:

1. Represent the game tree with terminal utility values.
2. Treat the AI as the maximizing player and the opponent as the minimizing player.
3. Recursively evaluate the game tree using the Minimax algorithm.
4. Display the evaluated values and the best move selected by the AI.
5. Explain how the algorithm assumes that both players choose optimal moves.

DIAGRAM – EXPERIMENT 7



PROGRAM

```
tree = [3, 5, 6, 9, 1, 2, 0, -1]
def minimax(values, depth, maximizing):
    if depth == 3:
```

```
        return values.pop(0)
    if maximizing:
        left = minimax(values, depth + 1, False)
        right = minimax(values, depth + 1, False)
        value = max(left, right)
        print("MAX node value:", value)
        return value
    else:
        left = minimax(values, depth + 1, True)
        right = minimax(values, depth + 1, True)
        value = min(left, right)
        print("MIN node value:", value)
        return value
# Use a copy because the recursive function consumes the list.
root_value = minimax(tree.copy(), 0, True)
print("Best utility for AI:", root_value)
print("Best move: Left subtree" if root_value == 5 else "Best move: Right subtree")
```

OUTPUT

```
MAX node value: 5
MAX node value: 9
MIN node value: 5
MAX node value: 2
MAX node value: 0
MIN node value: 0
MAX node value: 5
Best utility for AI: 5
Best move: Left subtree
```

RESULT

The Minimax algorithm evaluated the game tree and returned a best utility value of 5 for the AI. Based on this value, it selected the left subtree as the best move.

EXPERIMENT 8

Alpha-Beta Pruning for Game-Tree Search

AIM

To determine the best move in a binary game tree while reducing explored nodes using Alpha-Beta pruning.

QUESTION

A chess-playing AI evaluates possible moves by exploring a game tree. Since evaluating every possible move is computationally expensive, the AI uses Alpha-Beta Pruning to eliminate branches that cannot influence the final decision.

Develop a Python program to implement the Alpha-Beta Pruning algorithm for a chess-inspired game tree with multiple possible moves and terminal utility values.

The program must:

1. Represent the game tree with multiple candidate moves and terminal utility values.

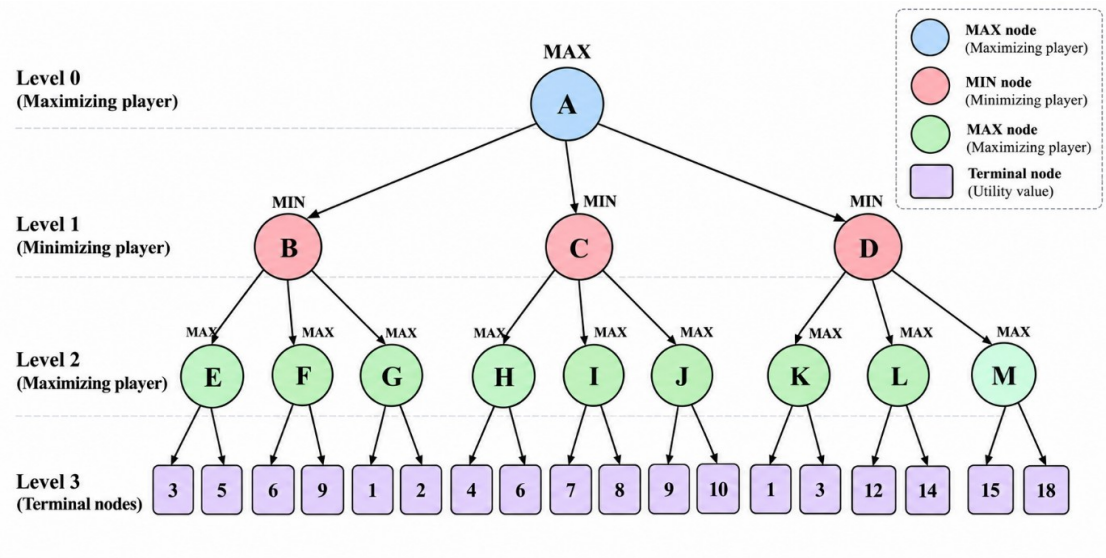
2. Implement Alpha-Beta Pruning using alpha and beta bounds.

3. Identify and skip branches that cannot affect the final decision.

4. Display the selected move, its utility value, the number of nodes evaluated, and the number of branches pruned.

5. Explain how Alpha-Beta Pruning reduces the search effort while preserving the optimal decision.

DIAGRAM – EXPERIMENT 8



PROGRAM

```
# Experiment-7:
# Binary game tree with eight terminal utility values
```

```
tree = [3, 5, 6, 9, 1, 2, 0, -1]
leaf_index = 0
nodes_evaluated = 0
branches_pruned = 0
def alpha_beta(depth, index, maximizing, alpha, beta):
    global nodes_evaluated, branches_pruned
    if depth == 3:
        nodes_evaluated += 1
        value = tree[index]
        print("Evaluated leaf:", value)
        return value
    if maximizing:
        value = float("-inf")
        for child in range(2):
            child_value = alpha_beta(
                depth + 1, index * 2 + child,
                False, alpha, beta
            )
            value = max(value, child_value)
            alpha = max(alpha, value)
            if alpha >= beta:
                branches_pruned += 1
                # Count the remaining child subtree as pruned.
                branches_pruned += 2 ** (3 - (depth + 1))
                break
        return value
    else:
        value = float("inf")
        for child in range(2):
            child_value = alpha_beta(
                depth + 1, index * 2 + child,
                True, alpha, beta
            )
            value = min(value, child_value)
            beta = min(beta, value)
            if alpha >= beta:
                # Remaining child subtree is skipped.
                branches_pruned += 2 ** (3 - (depth + 1))
                break
        return value
best_value = alpha_beta(0, 0, True, float("-inf"), float("inf"))
print("Best move: Left subtree" if best_value == 5 else "Best move: Right subtree")
print("Best utility value:", best_value)
print("Terminal nodes evaluated:", nodes_evaluated)
print("Branches/nodes skipped by pruning:", branches_pruned)
```

OUTPUT

```
Evaluated leaf: 3
Evaluated leaf: 5
```

Evaluated leaf: 6
Evaluated leaf: 1
Evaluated leaf: 2
Best move: Left subtree
Best utility value: 5
Terminal nodes evaluated: 5
Branches/nodes skipped by pruning: 4

RESULT

The Alpha-Beta Pruning program selected the left subtree with a best utility value of 5. It evaluated 5 terminal nodes and reported 4 branches or nodes skipped through pruning.

EXPERIMENT 9

Forward Chaining for Medical Appointment Eligibility

AIM

To infer medical appointment eligibility and related priorities from initial facts using forward chaining.

QUESTION

A healthcare appointment system uses facts and logical rules to determine whether a patient is eligible for a priority consultation. The system derives new facts from existing information by repeatedly applying rules until a conclusion is reached or no further facts can be inferred.

Develop a Python program to implement Forward Chaining using facts and rules.

The program must:

1. Represent the initial facts and inference rules.

2. Check whether the conditions of each rule are satisfied.

3. Infer and add new facts when a rule applies.

4. Continue the inference process until no new facts can be derived.

5. Display the initial facts, newly inferred facts, and final set of facts.

PROGRAM

```
# Experiment-8:
# Forward Chaining for Medical Appointment Eligibility

# Step 1: Define initial facts
facts = {
    "patient_registered",
    "has_fever",
    "has_high_blood_pressure",
    "emergency_symptoms"
}

# Step 2: Define inference rules
rules = [
    ({"patient_registered", "has_fever"}, "needs_medical_checkup"),
    ({"has_high_blood_pressure"}, "requires_monitoring"),
    ({"needs_medical_checkup", "requires_monitoring"},
     "priority_consultation"),
    ({"emergency_symptoms"}, "emergency_priority"),
    ({"priority_consultation", "patient_registered"},
     "eligible_for_priority_appointment")
]

# Step 3: Implement Forward Chaining
```

```
def forward_chaining(facts, rules):

    print("Initial Facts:")
    for fact in facts:
        print("-", fact)

    inferred_facts = set()
    new_fact_found = True

    while new_fact_found:
        new_fact_found = False

        for conditions, conclusion in rules:

            if conditions.issubset(facts):
                if conclusion not in facts:

                    facts.add(conclusion)
                    inferred_facts.add(conclusion)
                    new_fact_found = True

                    print("\nNewly Inferred Fact:", conclusion)

    # Step 4: Display final facts
    print("\nFinal Set of Facts:")
    for fact in facts:
        print("-", fact)

    print("\nNewly Inferred Facts:")
    for fact in inferred_facts:
        print("-", fact)


# Step 5: Execute the program
forward_chaining(facts, rules)
```

OUTPUT

```
Initial Facts:
- patient_registered
- has_fever
- emergency_symptoms
- has_high_blood_pressure

Newly Inferred Fact: needs_medical_checkup

Newly Inferred Fact: requires_monitoring

Newly Inferred Fact: priority_consultation
```

Newly Inferred Fact: emergency_priority

Newly Inferred Fact: eligible_for_priority_appointment

Final Set of Facts:
- has_fever
- priority_consultation
- emergency_priority
- eligible_for_priority_appointment
- needs_medical_checkup
- has_high_blood_pressure
- requires_monitoring
- patient_registered
- emergency_symptoms

Newly Inferred Facts:
- priority_consultation
- emergency_priority
- eligible_for_priority_appointment
- needs_medical_checkup
- requires_monitoring

RESULT

The forward-chaining system derived five new facts: needs_medical_checkup, requires_monitoring, priority_consultation, emergency_priority, and eligible_for_priority_appointment. The final fact set therefore includes the conclusion that the registered patient qualifies for a priority appointment under the defined rules.

EXPERIMENT 10

Forward Chaining for Priority Consultation

AIM

To infer whether a patient qualifies for a priority consultation by repeatedly applying logical rules using forward chaining.

QUESTION

A university models student well-being using logical facts and rules. It wants to determine whether a student is happy based on the conditions that the student enjoys learning and has sufficient free time.

Develop a Python program to implement Resolution Refutation to prove whether a student is happy.

The program must:

1. Represent the knowledge base using logical statements.

2. Convert the relevant statements into Conjunctive Normal Form (CNF).

3. Negate the goal to be proved and add it to the knowledge base.

4. Apply the resolution rule to derive new clauses.

5. Identify the empty clause, if obtained, to demonstrate a contradiction and establish the goal.

PROGRAM

```
#EX-9
facts = {
    "fever",
    "cough",
    "high_temperature",
    "age_over_65"
}

rules = [
    (["fever", "cough"], "respiratory_symptoms"),
    (["respiratory_symptoms", "high_temperature"],
     "suspected_infection"),
    (["suspected_infection", "age_over_65"],
     "priority_consultation")
]

trace = []

print("Forward Chaining Using Facts and Rules\n")

print("Initial Facts:")
```

```
for fact in sorted(facts):
    print("-", fact)

changed = True

while changed:
    changed = False

    for conditions, conclusion in rules:
        if all(condition in facts for condition in conditions):
            if conclusion not in facts:
                facts.add(conclusion)
                changed = True
                trace.append(
                    f"IF {conditions} THEN {conclusion}"
                )
                print(f"\nRule applied: {trace[-1]}")
                print(f"New fact inferred: {conclusion}")

print("\n--- Final Facts ---")
for fact in sorted(facts):
    print("-", fact)

print("\n--- Final Conclusion ---")
if "priority_consultation" in facts:
    print("Patient qualifies for a priority consultation.")
else:
    print("Priority consultation criteria not established.")

print("\nTotal new facts inferred:", len(trace))
```

OUTPUT

```
Forward Chaining Using Facts and Rules

Initial Facts:
- age_over_65
- cough
- fever
- high_temperature

Rule applied: IF ['fever', 'cough'] THEN respiratory_symptoms
New fact inferred: respiratory_symptoms

Rule applied: IF ['respiratory_symptoms', 'high_temperature'] THEN suspected_infection
New fact inferred: suspected_infection

Rule applied: IF ['suspected_infection', 'age_over_65'] THEN priority_consultation
New fact inferred: priority_consultation
```

--- Final Facts ---
- age_over_65
- cough
- fever
- high_temperature
- priority_consultation
- respiratory_symptoms
- suspected_infection

--- Final Conclusion ---
Patient qualifies for a priority consultation.

Total new facts inferred: 3

RESULT

The forward-chaining program inferred respiratory_symptoms, suspected_infection, and priority_consultation from the initial facts. It concluded that the patient qualifies for a priority consultation after applying three rules.

EXPERIMENT 11

Resolution Refutation to Prove Student Happiness

AIM

To prove whether a student is happy using resolution refutation based on the facts that the student enjoys learning and has free time.

QUESTION

Develop a program to represent family relationships using facts and rules. Given a family tree, identify the parents, children, and grandparents of specified individuals.

PROGRAM

```
#EX-10
clauses = [
    {"EnjoysLearning"},                 # C1
    {"HasFreeTime"},                    # C2
    {"NotEnjoysLearning", "Happy"},     # C3
    {"NotHasFreeTime", "Happy"}         # C4
]

goal = "Happy"

print("Resolution Refutation\n")

print("--- Knowledge Base (CNF) ---")

for i, clause in enumerate(clauses, start=1):
    print(f"C{i}: {' OR '.join(sorted(clause))}")

# Negate the goal and add it to the knowledge base

negated_goal = "Not" + goal

clauses.append({negated_goal})

print(f"\nNegated goal added: {negated_goal}")

# Resolve complementary literals

derived = set()

empty_clause_found = False

print("\n--- Resolution Steps ---")

changed = True
```

```
while changed and not empty_clause_found:

    changed = False

    current_clauses = list(clauses)

    for i in range(len(current_clauses)):

        for j in range(i + 1, len(current_clauses)):

            c1 = current_clauses[i]

            c2 = current_clauses[j]

            for literal in c1:

                if literal.startswith("Not"):
                    complement = literal[3:]
                else:
                    complement = "Not" + literal

                if complement in c2:

                    resolvent = (c1 - {literal}) | (c2 - {complement})

                    print(
                        f"Resolve {sorted(c1)} and {sorted(c2)} "
                        f"on {literal} -> {sorted(resolvent)}"
                    )

                    if not resolvent:
                        empty_clause_found = True
                        break

                    frozen_resolvent = frozenset(resolvent)

                    if (frozen_resolvent not in
                            {frozenset(c) for c in clauses} and
                        frozen_resolvent not in derived):

                        derived.add(frozen_resolvent)

                        changed = True

            if empty_clause_found:
                break

        if empty_clause_found:
            break
```

```
        for clause in derived:
            new_clause = set(clause)

            if new_clause not in clauses:
                clauses.append(new_clause)

        derived.clear()

    print("\n--- Final Result ---")

    if empty_clause_found:
        print("Empty clause derived: Yes")
        print("Contradiction found.")
        print("Therefore, the goal 'Happy' is proved.")

    else:
        print("Empty clause derived: No")
        print("The goal could not be proved by this resolution run.")
```

OUTPUT

```
Resolution Refutation

--- Knowledge Base (CNF) ---
C1: EnjoysLearning
C2: HasFreeTime
C3: Happy OR NotEnjoysLearning
C4: Happy OR NotHasFreeTime

Negated goal added: NotHappy

--- Resolution Steps ---
Resolve ['EnjoysLearning'] and ['Happy', 'NotEnjoysLearning'] on EnjoysLearning ->
['Happy']
Resolve ['HasFreeTime'] and ['Happy', 'NotHasFreeTime'] on HasFreeTime -> ['Happy']
Resolve ['Happy', 'NotEnjoysLearning'] and ['NotHappy'] on Happy -> ['NotEnjoysLearning']
Resolve ['Happy', 'NotHasFreeTime'] and ['NotHappy'] on Happy -> ['NotHasFreeTime']
Resolve ['EnjoysLearning'] and ['Happy', 'NotEnjoysLearning'] on EnjoysLearning ->
['Happy']
Resolve ['EnjoysLearning'] and ['NotEnjoysLearning'] on EnjoysLearning -> []

--- Final Result ---
Empty clause derived: Yes
Contradiction found.
Therefore, the goal 'Happy' is proved.
```

RESULT

The resolution-refutation program derived the empty clause after adding NotHappy to the

knowledge base. This contradiction establishes Happy from the encoded facts and rules.

EXPERIMENT 12

Family Tree Relationships Using Logic

AIM

To represent family relationships and answer parent, child and grandparent queries using a logical fact base.

QUESTION

A healthcare support system receives a patient's description of symptoms in natural language. The system must identify relevant symptom keywords and phrases, match them against predefined rules, and return suitable guidance or recommend professional assessment.

Develop a Python program that applies pattern matching to patient-reported symptoms and identifies predefined symptom patterns to generate relevant guidance.

PROGRAM

```
#EX-11
parents = [
    ("Ravi", "Arun"),
    ("Ravi", "Meena"),
    ("Arun", "Kiran"),
    ("Meena", "Priya"),
    ("Suresh", "Ravi")
]

def is_parent(parent, child):
    return (parent, child) in parents

def get_children(parent):
    return [child for p, child in parents if p == parent]

def get_parents(child):
    return [parent for parent, c in parents if c == child]

def get_grandparents(person):
    grandparents = []

    for parent in get_parents(person):
        grandparents.extend(get_parents(parent))

    return grandparents

# Display family relationships

person = "Kiran"

print("Family Tree Relationships")
```

```
print("-------------------------")

print("Parents of", person, ":", get_parents(person))

print("Children of Ravi:", get_children("Ravi"))

print("Grandparents of", person, ":", get_grandparents(person))

print("\nIs Ravi a parent of Arun?", is_parent("Ravi", "Arun"))

print("Is Suresh a grandparent of Kiran?",
    "Suresh" in get_grandparents("Kiran"))
```

OUTPUT

```
Family Tree Relationships
-------------------------
Parents of Kiran : ['Arun']
Children of Ravi: ['Arun', 'Meena']
Grandparents of Kiran : ['Ravi']

Is Ravi a parent of Arun? True
Is Suresh a grandparent of Kiran? False
```

RESULT

The family-tree program identified Arun as Kiran’s parent, Arun and Meena as Ravi’s children, and Ravi as Kiran’s grandparent. It also returned True for Ravi being Arun’s parent and False for Suresh being Kiran’s grandparent, according to the relationships defined in the program.

EXPERIMENT 13

Healthcare Symptom Assessment Using Pattern Matching

AIM

To identify symptom categories from a patient's text input using predefined pattern matching and provide corresponding guidance.

QUESTION

A healthcare research system uses tumour measurement data to classify samples into two categories: benign and malignant. An Artificial Neural Network can learn patterns from previously labelled samples by adjusting its weights and biases through backpropagation.

Develop a Python program to implement an Artificial Neural Network using backpropagation.

The program must:

1. Load and preprocess the breast cancer dataset.

2. Construct a neural network with an input layer, one hidden layer, and an output layer.

3. Use the sigmoid activation function.

4. Train the network using forward propagation and backpropagation.

5. Predict the classification of unseen samples.

6. Evaluate the model using classification accuracy and a confusion matrix.

PROGRAM

```
#EX-12
import re

# Healthcare symptom patterns

symptom_patterns = {

    "Emergency Warning Signs": [
        r"\bchest pain\b",
        r"\bdifficulty breathing\b",
        r"\bshortness of breath\b",
        r"\bface drooping\b",
        r"\bsudden weakness\b",
        r"\bsevere bleeding\b"
    ],

    "Respiratory Symptoms": [
        r"\bcough\b",
        r"\bsore throat\b",
        r"\brunny nose\b",
        r"\bfever\b"
```

```
    ],

    "Digestive Symptoms": [
        r"\bnausea\b",
        r"\bvomiting\b",
        r"\bstomach pain\b",
        r"\babdominal pain\b",
        r"\bdiarrhea\b"
    ],

    "Neurological or General Symptoms": [
        r"\bheadache\b",
        r"\bdizziness\b",
        r"\bfatigue\b",
        r"\bweakness\b"
    ]

}

# Guidance associated with matched patterns

guidance = {

    "Emergency Warning Signs":
        "Seek emergency medical care immediately. Do not delay care "
        "while using this system.",

    "Respiratory Symptoms":
        "Consider professional medical advice if symptoms persist, "
        "worsen, or are accompanied by breathing difficulty.",

    "Digestive Symptoms":
        "Maintain hydration if appropriate and seek medical advice "
        "for persistent or worsening symptoms.",

    "Neurological or General Symptoms":
        "Consider medical assessment if symptoms persist or worsen. "
        "Sudden weakness or other sudden neurological changes require "
        "emergency assessment."

}


def match_symptoms(patient_input):

    text = patient_input.lower()

    matched_categories = []
    matched_symptoms = []
```

```
    for category, patterns in symptom_patterns.items():

        category_matches = []

        for pattern in patterns:

            if re.search(pattern, text):
                category_matches.append(
                    re.search(pattern, text).group()
                )

        if category_matches:
            matched_categories.append(category)
            matched_symptoms.extend(category_matches)

    return matched_categories, matched_symptoms


def healthcare_assistant():

    print("HEALTHCARE SYMPTOM ASSESSMENT SYSTEM")
    print("-" * 40)

    print("Please enter your symptoms:")
    print("Example: Fever, cough, headache and sore throat.")

    patient_input = input("\nEnter your symptoms: ").strip()

    if not patient_input:

        print("System: Please enter a symptom description.")
        return

    categories, symptoms = match_symptoms(patient_input)

    print("\nASSESSMENT REPORT")
    print("-" * 40)

    if not categories:

        print("Matched symptoms: None")
        print("System: No predefined pattern was matched.")

        print("Please describe the symptoms more clearly or "
              "consult a healthcare professional.")

        return
```

```
    print("Matched symptoms:", ", ".join(sorted(set(symptoms))))
    print("Matched categories:", ", ".join(categories))

    # Emergency patterns take priority over other matches

    if "Emergency Warning Signs" in categories:

        print("\nPriority: EMERGENCY WARNING SIGN")
        print("Guidance:", guidance["Emergency Warning Signs"])

    else:

        print("\nPriority: NON-EMERGENCY PATTERN MATCH")

        for category in categories:
            print(f"{category}: {guidance[category]}")

    print("\nNote: This system does not diagnose diseases "
          "or replace professional medical assessment.")


healthcare_assistant()
```

OUTPUT

```
HEALTHCARE SYMPTOM ASSESSMENT SYSTEM
----------------------------------------
Please enter your symptoms:
Example: Fever, cough, headache and sore throat.

Enter your symptoms: Fever, cough, headache and sore throat

ASSESSMENT REPORT
----------------------------------------
Matched symptoms: cough, fever, headache, sore throat
Matched categories: Respiratory Symptoms, Neurological or General Symptoms

Priority: NON-EMERGENCY PATTERN MATCH
Respiratory Symptoms: Consider professional medical advice if symptoms persist, worsen, or
are accompanied by breathing difficulty.
Neurological or General Symptoms: Consider medical assessment if symptoms persist or
worsen. Sudden weakness or other sudden neurological changes require emergency
assessment.

Note: This system does not diagnose diseases or replace professional medical assessment.
```

RESULT

The pattern-matching system identified cough, fever, headache, and sore throat, classifying them as respiratory and neurological/general symptoms. It displayed non-emergency guidance

and stated that the system does not diagnose diseases or replace professional medical assessment.

EXPERIMENT 14

Artificial Neural Network Using Backpropagation

AIM

To implement and train a feed-forward artificial neural network using backpropagation for breast cancer classification.

QUESTION

An automated postal processing system needs to recognize handwritten digits on envelopes to identify PIN codes and route mail to the correct locations. A neural network can learn patterns from handwritten digit images and classify each image into one of ten digit classes (0–9).

Develop an artificial neural network that classifies handwritten digit images into ten categories (0–9). Use the MNIST dataset, preprocess the image data, train the network using backpropagation, and evaluate its classification performance.

PROGRAM

```
#EX-13

import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target.reshape(-1, 1)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Neural network architecture
input_neurons = X_train.shape[1]
hidden_neurons = 8
output_neurons = 1

np.random.seed(42)
```

```
W1 = np.random.randn(input_neurons, hidden_neurons) * np.sqrt(2 / input_neurons)
b1 = np.zeros((1, hidden_neurons))

W2 = np.random.randn(hidden_neurons, output_neurons) * np.sqrt(2 / hidden_neurons)
b2 = np.zeros((1, output_neurons))

# Sigmoid activation function
def sigmoid(x):
    x = np.clip(x, -500, 500)
    return 1 / (1 + np.exp(-x))

# Training parameters
learning_rate = 0.05
epochs = 3000

# Backpropagation training
for epoch in range(epochs):

    hidden_input = np.dot(X_train, W1) + b1
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2) + b2
    predicted_output = sigmoid(final_input)

    output_delta = (predicted_output - y_train) / len(y_train)

    dW2 = np.dot(hidden_output.T, output_delta)
    db2 = np.sum(output_delta, axis=0, keepdims=True)

    hidden_delta = (
        np.dot(output_delta, W2.T)
        * hidden_output * (1 - hidden_output)
    )

    dW1 = np.dot(X_train.T, hidden_delta)
    db1 = np.sum(hidden_delta, axis=0, keepdims=True)

    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2
    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1

# Testing the neural network
hidden_test = sigmoid(np.dot(X_test, W1) + b1)
test_output = sigmoid(np.dot(hidden_test, W2) + b2)

y_pred = (test_output >= 0.5).astype(int)

# Model evaluation
```

```
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("Artificial Neural Network using Backpropagation")
print("----------------------------------------------")
print("Input neurons:", input_neurons)
print("Hidden neurons:", hidden_neurons)
print("Output neurons:", output_neurons)
print("Training epochs:", epochs)

print("\nClassification Accuracy:",
      round(accuracy * 100, 2), "%")

print("\nConfusion Matrix:")
print(cm)

print("\nClass Labels:")
print("0 - Malignant")
print("1 - Benign")
```

OUTPUT

```
Artificial Neural Network using Backpropagation
----------------------------------------------
Input neurons: 30
Hidden neurons: 8
Output neurons: 1
Training epochs: 3000

Classification Accuracy: 95.61 %

Confusion Matrix:
[[41  1]
 [ 4 68]]

Class Labels:
0 - Malignant
1 - Benign
```

RESULT

The artificial neural network trained for 3,000 epochs and reported a classification accuracy of 95.61% on the breast-cancer test set. The confusion matrix was [[41, 1], [4, 68]], with class 0 representing malignant samples and class 1 representing benign samples.

EXPERIMENT 15

Handwritten Digit Recognition Using an Artificial Neural Network

AIM

To build and evaluate an artificial neural network for handwritten digit recognition using a 10-class output layer.

QUESTION

Develop a binary classification model using the Breast Cancer Wisconsin Diagnostic dataset. Train a perceptron using tumour measurements and classify samples into benign and malignant categories. Evaluate the model using accuracy and a confusion matrix.

PROGRAM

```
#Exper-14:
import numpy as np
import tensorflow as tf

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Set random seeds
np.random.seed(42)
tf.random.set_seed(42)

# Generate sample data
x_train = np.random.rand(1000, 784).astype("float32")
y_train = np.random.randint(0, 10, 1000)

x_test = np.random.rand(200, 784).astype("float32")
y_test = np.random.randint(0, 10, 200)

# One-hot encoding
y_train_encoded = tf.keras.utils.to_categorical(y_train, 10)
y_test_encoded = tf.keras.utils.to_categorical(y_test, 10)

# Build ANN model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(784,)),
    tf.keras.layers.Dense(128, activation="sigmoid"),
    tf.keras.layers.Dense(10, activation="softmax")
])

# Compile model
model.compile(
```

```
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

print("Handwritten Digit Recognition")
print("--------------------------------")

model.summary()

# Train the model
history = model.fit(
    x_train,
    y_train_encoded,
    epochs=5,
    batch_size=128,
    validation_split=0.1,
    verbose=1
)

# Predict test data
predicted_probabilities = model.predict(x_test, verbose=0)

y_pred = np.argmax(predicted_probabilities, axis=1)

# Evaluate model
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("\nModel Evaluation")
print("--------------------------------")

print("Testing Accuracy: {:.2f}%".format(accuracy * 100))

print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
```

OUTPUT

The uploaded notebook does not contain a saved execution output for this experiment. The program requires TensorFlow. TensorFlow was not installed in the execution environment, so a genuine output could not be generated here. Run the program in an environment with TensorFlow installed.

RESULT

A complete execution result is not available for this experiment because the program’s output

was not saved and TensorFlow was unavailable in the execution environment. The program must be run in an environment with TensorFlow installed to obtain its training results, test accuracy, confusion matrix, and classification report.

EXPERIMENT 16

Breast Cancer Classification Using Perceptron

AIM

To train a Perceptron model and classify breast cancer samples as benign or malignant.

QUESTION

In the food and beverage industry, laboratory measurements can help identify wine categories based on their chemical properties. A machine learning model can analyse features such as alcohol content, acidity, magnesium and flavonoids to classify wine samples into their respective categories.

Develop a classification system using the Wine Recognition dataset. Train a Gaussian Naïve Bayes model using the chemical measurements of wine samples and predict their respective categories. Evaluate the model using classification accuracy and a confusion matrix.

PROGRAM

```
# EXP-15: Perceptron - Breast Cancer Classification


from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Load Dataset
data = load_breast_cancer()

X = data.data
y = data.target

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Preprocessing
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Create Perceptron Model
model = Perceptron(
    max_iter=1000,
    eta0=0.01,
    random_state=42
)

# Train Model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation
accuracy = accuracy_score(y_test, y_pred)

# Display Output
print("Breast Cancer Classification Using Perceptron")
print("-----------------------------------------------")

print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))

print("\nClassification Accuracy: {:.2f}%".format(
    accuracy * 100
))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred, labels=[1, 0]))

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    labels=[1, 0],
    target_names=["Benign", "Malignant"]
))

# New Sample Prediction
sample = X_test[0].reshape(1, -1)

prediction = model.predict(sample)

print("New Sample Prediction:",
      "Benign" if prediction[0] == 1 else "Malignant")
```

OUTPUT

```
Breast Cancer Classification Using Perceptron
-----------------------------------------------
```

Training Samples: 455
Testing Samples: 114

Classification Accuracy: 99.12%

Confusion Matrix:
[[71 0]
 [1 42]]

Classification Report:

	precision	recall	f1-score	support
Benign	0.99	1.00	0.99	71
Malignant	1.00	0.98	0.99	43
accuracy			0.99	114
macro avg	0.99	0.99	0.99	114
weighted avg	0.99	0.99	0.99	114

New Sample Prediction: Benign

RESULT

The Perceptron program classified the breast-cancer dataset using 455 training samples and 114 testing samples. It reported 99.12% classification accuracy and predicted the displayed new sample as Benign.

AI LAB – 5

Experiment 17: ID3 Decision Tree for Loan Approval Classification

AIM: To implement automated loan eligibility assessment in banking using python

QUESTION

A bank receives loan applications from customers with different financial backgrounds. A decision-support system can analyse customer information, such as income, employment status, credit history and existing loans, to classify applications as approved or rejected.

Develop a Python program that implements the ID3 algorithm using a customer loan dataset. The program should:

- Create a dataset containing customer financial attributes.
- Calculate entropy and information gain for each attribute.
- Select the attribute with the highest information gain.
- Construct a decision tree recursively.
- Classify new loan applications using the generated decision tree.

CODE:

```
import math
import collections
data = [
    ["Good", "High", "Salaried", "No", "Approved"],
    ["Good", "High", "Self-employed", "No", "Approved"],
    ["Good", "Medium", "Salaried", "No", "Approved"],
    ["Good", "Low", "Salaried", "No", "Approved"],
    ["Good", "Medium", "Self-employed", "Yes", "Approved"],
    ["Good", "Low", "Self-employed", "No", "Approved"],
    ["Poor", "High", "Salaried", "No", "Approved"],
    ["Poor", "Medium", "Salaried", "No", "Approved"],
    ["Poor", "Low", "Salaried", "No", "Rejected"],
    ["Poor", "High", "Self-employed", "Yes", "Rejected"],
    ["Poor", "Medium", "Self-employed", "No", "Rejected"],
    ["Poor", "Low", "Self-employed", "Yes", "Rejected"],
    ["Good", "High", "Salaried", "Yes", "Rejected"],
    ["Good", "Medium", "Self-employed", "Yes", "Rejected"],
```

```
    ["Poor", "High", "Salaried", "Yes", "Rejected"],
    ["Poor", "Low", "Salaried", "Yes", "Rejected"]
]
features = [
    "Credit History",
    "Income",
    "Employment",
    "Existing Loan"
]
def calculate_entropy(rows):
    labels = [row[-1] for row in rows]
    counts = collections.Counter(labels)
    total = len(labels)
    entropy = 0
    for count in counts.values():
        probability = count / total
        entropy -= probability * math.log2(probability)
    return entropy
def information_gain(rows, feature_index):
    total_entropy = calculate_entropy(rows)
    groups = collections.defaultdict(list)
    for row in rows:
        groups[row[feature_index]].append(row)
    weighted_entropy = 0
    for group in groups.values():
        weighted_entropy += (
            len(group) / len(rows)
        ) * calculate_entropy(group)
    return total_entropy - weighted_entropy
def build_tree(rows, available_features):
    labels = [row[-1] for row in rows]
```

```
    if len(set(labels)) == 1:
        return labels[0]
    if not available_features:
        return collections.Counter(labels).most_common(1)[0][0]
    best_feature = max(
        available_features,
        key=lambda index: information_gain(rows, index)
    )
    tree = {features[best_feature]: {}}
    values = sorted(set(row[best_feature] for row in rows))
    for value in values:
        subset = [
            row for row in rows
            if row[best_feature] == value
        ]
        remaining_features = [
            index for index in available_features
            if index != best_feature
        ]
        tree[features[best_feature]][value] = build_tree(
            subset, remaining_features
        )
    return tree
def predict(tree, sample):

    while isinstance(tree, dict):
        attribute = next(iter(tree))
        value = sample.get(attribute)
        branches = tree[attribute]
        if value not in branches:
            return "Unknown"
```

```
        tree = branches[value]
    return tree
initial_entropy = calculate_entropy(data)
print("ID3 Decision Tree - Loan Approval")
print("----------------------------------")
print("\nInitial Entropy:", round(initial_entropy, 4))
print("\nInformation Gain:")
for index, feature in enumerate(features):
    gain = information_gain(data, index)
    print(feature, ":", round(gain, 4))
tree = build_tree(data, list(range(len(features))))
print("\nDecision Tree Root:")
print(next(iter(tree)))
samples = [
    {
        "Credit History": "Good",
        "Income": "High",
        "Employment": "Salaried",
        "Existing Loan": "No"
    },
    {
        "Credit History": "Poor",
        "Income": "Low",
        "Employment": "Self-employed",
        "Existing Loan": "Yes"
    },
    {
        "Credit History": "Poor",
        "Income": "High",
        "Employment": "Salaried",
        "Existing Loan": "No"
```

```
    }
]
print("\nLoan Application Predictions:")

for i, sample in enumerate(samples, 1):
    result = predict(tree, sample)
    print("Application", i, ":", result)
```

OUTPUT:

```
ID3 Decision Tree - Loan Approval
---------------------------------
Initial Entropy: 1.0
Information Gain:
Credit History : 0.1887
Income : 0.0182
Employment : 0.0115
Existing Loan : 0.3113
Decision Tree Root:
Existing Loan

Loan Application Predictions:
Application 1 : Approved
Application 2 : Rejected
Application 3 : Approved
```

RESULT

The ID3 decision tree calculated an initial entropy of 1.0 and identified Existing Loan as the root attribute because it had the highest information gain (0.3113). The model classified Applications 1 and 3 as Approved and Application 2 as Rejected, demonstrating the use of information gain to support loan-approval classification.

Experiment 18: Monkey and Banana Problem Using State-Space Planning

AIM: To implement Goal-based planning in an autonomous warehouse robot using python.

QUESTION

A monkey is inside a room containing a box and a banana hanging from the ceiling. The monkey cannot reach the banana directly. It must move to the box, push it beneath the banana, climb onto the box and grasp the banana.

Develop a Python program that represents the problem using state-space representation and determines the sequence of actions required to obtain the banana.

CODE:

```
from collections import deque

# Locations: 0 = A, 1 = B, 2 = C
# Banana is hanging at location C

initial_state = (0, 1, False, False)
goal_location = 2

# State: (monkey_position, box_position, on_box, has_banana)

def get_actions(state):
    monkey, box, on_box, has_banana = state
    actions = []

    if has_banana:
        return actions

    # Move monkey left or right
    if not on_box:
        for direction in [-1, 1]:
            new_position = monkey + direction
```

```
        if 0 <= new_position <= 2 and new_position != box:
            actions.append((
                (new_position, box, False, False),
                "Move to location " + str(new_position)
            ))

# Push the box one step away from the monkey
if not on_box and abs(monkey - box) == 1:
    direction = 1 if box > monkey else -1
    new_box_position = box + direction

    if 0 <= new_box_position <= 2:
        actions.append((
            (box, new_box_position, False, False),
            "Push the box to location " + str(new_box_position)
        ))

# Climb onto the box
if not on_box and abs(monkey - box) == 1 and box == goal_location:
    actions.append((
        (box, box, True, False),
        "Climb onto the box"
    ))

# Grasp the banana
if on_box and monkey == goal_location and box == goal_location:
    actions.append((
        (monkey, box, True, True),
        "Grasp the banana"
    ))
```

```
    return actions

def solve_monkey_problem():
    queue = deque([(initial_state, [])])
    visited = {initial_state}

    while queue:
        current_state, path = queue.popleft()

        if current_state[3]:
            return path

        for next_state, action in get_actions(current_state):
            if next_state not in visited:
                visited.add(next_state)
                queue.append((next_state, path + [action]))

    return None

# Execute the planning algorithm
solution = solve_monkey_problem()

print("Monkey and Banana Problem")
print("-------------------------")
print("Initial State: Monkey at A, Box at B")
print("Banana Location: C")
print("\nSequence of Actions:")

if solution:
    for step, action in enumerate(solution, 1):
        print("Step", step, ":", action)
```

```
    print("\nGoal Achieved: Monkey obtained the banana!")
    print("Total Actions:", len(solution))
else:
    print("No solution found.")
```

OUTPUT:

```
Monkey and Banana Problem
-------------------------
Initial State: Monkey at A, Box at B
Banana Location: C
Sequence of Actions:
Step 1 : Push the box to location 2
Step 2 : Climb onto the box
Step 3 : Grasp the banana
Goal Achieved: Monkey obtained the banana!
Total Actions: 3
```

RESULT

The state-space planning program successfully generated a sequence of three actions: push the box to location C, climb onto the box, and grasp the banana. The goal state was reached, confirming that the monkey obtained the banana using the planned action sequence.

Experiment 19: Towers of Hanoi Using Recursion in Python

AIM: To implement automated disk transfer and robotic manipulation using python.

QUESTION

A robotic system needs to transfer a stack of objects from one platform to another while following specific placement constraints. The Towers of Hanoi problem demonstrates how a system can break a complex task into smaller subproblems using recursion.

Three rods are provided: Source (A), Auxiliary (B) and Destination (C). A stack of disks of different sizes is placed on the source rod.

Develop a Python program to transfer all disks from the source rod to the destination rod while following these rules:

- Only one disk can be moved at a time.

- Only the top disk of a rod can be moved.
- A larger disk cannot be placed on a smaller disk.

CODE:

```
move_count = 0
def tower_of_hanoi(n, source, auxiliary, destination):
    global move_count
    if n == 1:
        move_count += 1
        print(
            "Move disk 1 from",
            source,
            "to",
            destination
        )
        return
    tower_of_hanoi(
        n - 1,
        source,
        destination,
        auxiliary
    )
    move_count += 1
    print(
        "Move disk", n,
        "from", source,
        "to", destination
    )
    tower_of_hanoi(
        n - 1,
        auxiliary,
```

```
        source,
        destination
    )
n = 3
print("Towers of Hanoi")
print("----------------")
print("Number of disks:", n)
print("\nSequence of moves:")
tower_of_hanoi(n, 'A', 'B', 'C')
print("\nTotal Moves:", move_count)
print("Minimum Moves Required:", 2**n - 1)
```

OUTPUT:

```
Towers of Hanoi
----------------
Number of disks: 3
Sequence of moves:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

Total Moves: 7
Minimum Moves Required: 7
```

RESULT

The recursive Towers of Hanoi program transferred all three disks from rod A to rod C while following the movement rules. It completed the task in 7 moves, which matches the minimum number of moves required for three disks.

Experiment 20: Simple Reflex Agent for Vacuum World

AIM: To implement autonomous cleaning robot for smart offices using python.

QUESTION:

A smart office uses an autonomous vacuum-cleaning robot to clean different rooms. The robot senses whether its current location is dirty or clean and selects an appropriate action, such as cleaning the floor or moving to another location.

Develop a Python program that simulates a vacuum-cleaning agent operating in a four-room environment.

The agent should:

- Identify its current location.
- Detect whether the current room is dirty or clean.
- Clean the room if it is dirty.
- Move to the next room if the current room is clean.
- Stop the simulation when all rooms are clean.

CODE:

```
rooms = {
   'A': 'Dirty',
   'B': 'Clean',
   'C': 'Dirty',
   'D': 'Dirty'
}
locations = ['A', 'B', 'C', 'D']
current_position = 0
def simple_reflex_agent(location, status):
   if status == 'Dirty':
      return 'Suck'
   return 'Move'
print("Simple Reflex Agent - Vacuum World")
print("-----------------------------------")
print("Initial Room Conditions:", rooms)
step = 1
```

```
while True:
    if all(status == 'Clean' for status in rooms.values()):
        print("\nAll rooms are clean!")
        print("Cleaning task completed.")
        break
    current_room = locations[current_position]
    current_status = rooms[current_room]
    action = simple_reflex_agent(
        current_room,
        current_status
    )
    print(
        "\nStep", step,
        "- Location:", current_room,
        "- Status:", current_status
    )

    # Execute action
    if action == 'Suck':
        rooms[current_room] = 'Clean'
        print("Action: Cleaning room", current_room)
    else:
        print("Action: Moving to next room")
        current_position = (
            current_position + 1
        ) % len(locations)
    step += 1
print("\nFinal Room Conditions:", rooms)
print("Total Actions Performed:", step - 1)
```

OUTPUT:

Simple Reflex Agent - Vacuum World

Initial Room Conditions: {'A': 'Dirty', 'B': 'Clean', 'C': 'Dirty', 'D': 'Dirty'}

Step 1 - Location: A - Status: Dirty

Action: Cleaning room A

Step 2 - Location: A - Status: Clean

Action: Moving to next room

Step 3 - Location: B - Status: Clean

Action: Moving to next room

Step 4 - Location: C - Status: Dirty

Action: Cleaning room C

Step 5 - Location: C - Status: Clean

Action: Moving to next room

Step 6 - Location: D - Status: Dirty

Action: Cleaning room D

All rooms are clean!

Cleaning task completed.

Final Room Conditions: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean', 'D': 'Clean'}

Total Actions Performed: 6

RESULT

The simple reflex agent successfully cleaned all four rooms. It cleaned rooms A, C, and D when they were dirty, moved through the clean rooms, and completed the task with every room marked Clean after 6 actions.